



Aras Innovator 33

Domain Access Control Guide

Document #: D-008102

Last Modified: 8/22/2024

Copyright Information

Copyright © 2024 Aras Corporation. All Rights Reserved.

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Phone: 978-691-8900

E-mail: support@aras.com

Website: <https://www.aras.com/>

Notice of Rights

Copyright © 2024 by Aras Corporation and/or its affiliates. All rights reserved.

This document is protected by U.S. and international copyright laws and conventions. No copyright may be obscured or removed from this document. This document may not be modified or altered, or reproduced or transmitted in any form, without the explicit permission of the copyright holder.

Aras Innovator, Aras, and the Aras Corp "A" logo are registered trademarks of Aras Corporation in the United States and other countries.

All other trademarks referenced herein are the property of their respective owners.

Notice of Liability

THIS DOCUMENT IS PROVIDED FOR INFORMATIONAL PURPOSES ONLY, AND THE CONTENTS HEREOF ARE SUBJECT TO CHANGE WITHOUT NOTICE. THE INFORMATION CONTAINED IN THIS DOCUMENT IS DISTRIBUTED ON AN "AS IS" BASIS, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE OR A WARRANTY OF NON-INFRINGEMENT. ARAS SHALL HAVE NO LIABILITY TO ANY PERSON OR ENTITY WITH RESPECT TO ANY LOSS OR DAMAGE CAUSED OR ALLEGED TO BE CAUSED DIRECTLY OR INDIRECTLY BY THE INFORMATION CONTAINED IN THIS DOCUMENT OR BY THE SOFTWARE OR HARDWARE PRODUCTS DESCRIBED HEREIN.

Table of Contents

Send Us Your Comments 4

1 Overview 5

1.1 Relationship Structures as Access Control Domains 5

1.1.1 Query Definition 5

1.2 DAC Subdomains 6

1.2.1 DAC Subdomain Rules 6

1.2.2 Mapping Team Roles from the Root Item Team 9

1.3 Domain Access Control (DAC) vs. RBAC and MAC 9

1.4 DAC Data Model Considerations 10

1.4.1 Tracking Changes to Access Controlled Items 11

1.4.2 Customizing DRF Event Handlers 12

1.5 Terminology 12

2 Creating and Activating a DAC Domain Policy 14

2.1 Prerequisites 14

2.2 Creating a Query Definition 16

2.3 Derived Relationship Families 16

2.3.1 Creating a Derived Relationship Family 17

2.3.2 Active Derived Relationship Families 22

2.3.3 Deactivating a Derived Relationship Family 22

2.3.4 Derived Relationship Family Remediation 24

2.4 DAC Definitions 26

2.5 Creating Subdomain Permissions 27

2.6 Creating a DAC Definition 29

2.7 Activating/Deactivating a DAC Definition 32

Appendix 33

Creating Teams 33

Creating Projects 34

Associating Variables with a Project 37

Custom DRF Events Handling 40

The dr_EnsurePropagatedAccess Event Handler 41

Send Us Your Comments

Aras Corporation welcomes your comments and suggestions on the quality and usefulness of this document. Your input is an important part of the information used for future revisions.

- Did you find any errors?
- Is the information clearly presented?
- Do you need more information? If so, where and what level of detail?
- Are the examples correct? Do you need more examples?
- What features did you like most?

If you find any errors or have any other suggestions for improvement, indicate the document title, and the chapter, section, and page number (if available).

You can send comments to us in the following ways:

Email:

TechDocs@aras.com

Subject: Aras Product Documentation

Or,

Postal service:

Aras Corporation
100 Brickstone Square
Suite 100
Andover, MA 01810
Attention: Aras Technical Documentation

If you would like a reply, provide your name, email address, address, and telephone number.

If you have usage issues with the software, visit <https://www.aras.com/support/>

1 Overview

Domain Access Control (DAC) is an advanced form of access control that gives Administrators the ability to create relationship-based access controls within Aras Innovator. This type of access control can be used to grant access to specific data within the scope of a “project” or “workspace”.

DAC works in conjunction with standard Aras Innovator role-based permissions and MAC policies. In order to have access to an item, users must have permissions through either DAC or standard Aras Innovator role-based permissions. In addition, a user must satisfy any applicable MAC Policies.

1.1 Relationship Structures as Access Control Domains

DAC grants access to items based on an item’s occurrence in a relationship structure under a Root item. The Root of the structure is the Domain scope—**Project X** at right. Access to related **Document** item can be controlled by DAC for the Project Domain. DAC can grant or elevate access to the Items within its domain.

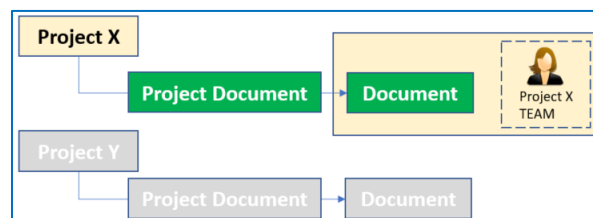


Figure 1.

DAC uses a ‘derived’ representation of existing physical relationships without making any changes to the source items, related items or relationships themselves. Derived Relationships allow DAC to track changes and organize access rules against the corresponding business data.

One or more branches or paths from a Root to a related item can be used to derive relationships. Each path and its endpoints define a DAC Subdomain. A DAC Domain contains all of the Subdomains derived by one Query Definition against a Root item.

1.1.1 Query Definition

A standard Aras Query Definition is used to indicate which parts of an overall ItemType Relationship structure is to be used by DAC in an access control policy.

Specifically, a query is used to extract the relationship ‘paths’ from Root to Leaf items that will implement DAC access control. The Aras Query Builder is used to define a Query Definition for DAC.

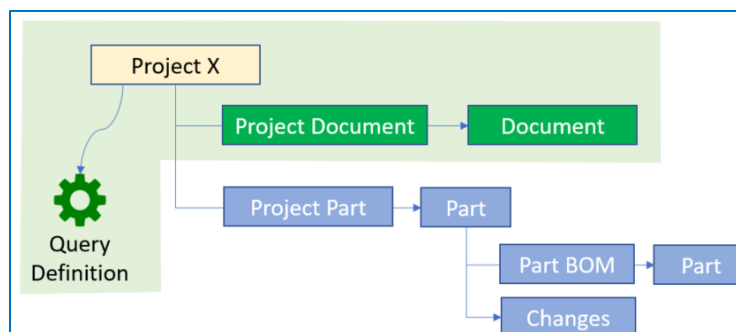


Figure 2.

Query Conditions, where-used references, and other Query Builder features are supported, thus providing a powerful method of identifying the specific paths, or 'Subdomains' within the Relationship structure where Access Rules will be set by DAC.

1.2 DAC Subdomains

Each branch or path within a Relationship structure can be used to implement a discreet set of Access Rules in a DAC **Subdomain**. There must be at least one Subdomain in any DAC definition.

In the following example **Project** is the root of the Domain and two subdomains are configured under DAC access control – *Subdomain_1* and *Subdomain_2*. Both subdomains control the **Document** ItemType in this case, but any relationship can be used. *Subdomain_1* has an intermediate **Part** node and enforces different access rules than *Subdomain_2* for the same Document ItemType which is related directly to the Project.

The Document under Part can gain an elevated access for Engineering Roles via DAC, whereas DAC can grant 'Get' access to all members of the Project Team to Documents related to the Project directly, depending on business requirements.

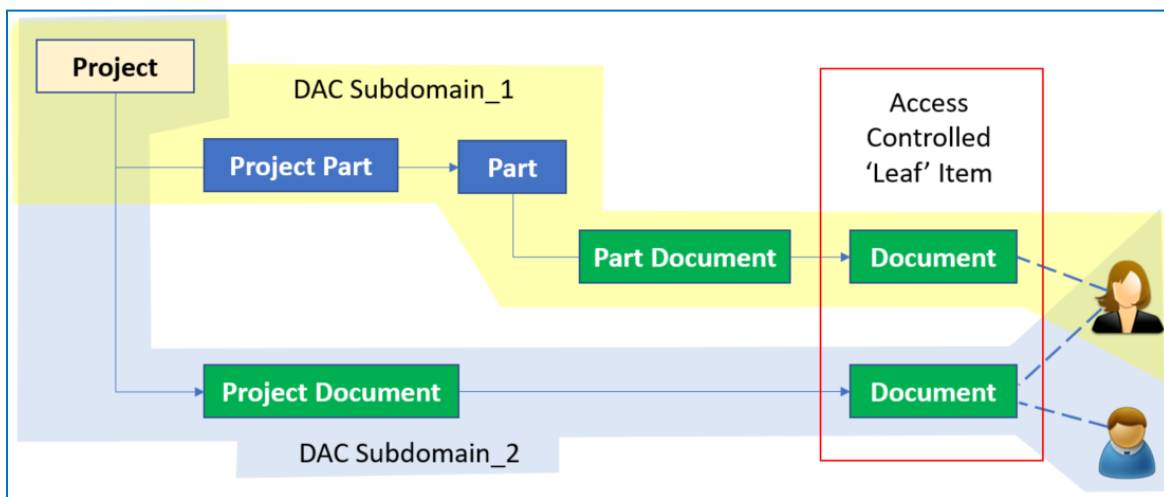


Figure 3.

1.2.1 DAC Subdomain Rules

For each subdomain used in a DAC Definition, the administrator must define one or more Subdomain Rules to ensure that the correct permissions are granted to the Leaf item. A subdomain rule determines the proper permissions to grant based on input of the fields shown in Figure 4 and described in Table 2.

Project → Project Docs Part Docs	Part Docs		
	No Related		
Priority	Subdomain Permission [...]	Root Permission [...]	Destination Life Cycle State [...]

Figure 4.

Table 1: Fields for granting permissions

Field	Description	Required?
Priority	Used to set the order in which the Rules are evaluated. This is useful when the same Subdomain Permission and Root Permission pair appear in more than one row, but with a different (or blank) Life Cycle State (see detail below).	Yes
Subdomain Permission	Permission to grant to Leaf Item if Rule is satisfied.	Yes
Root Permission	Permission to match with the current permission on the Root Item (e.g. Project).	Yes
Destination Life Cycle State	If specified it forces a match with the current Leaf Item lifecycle state.	Optional

When the DAC Policy is active the fields are evaluated at runtime as shown in Figure 5.

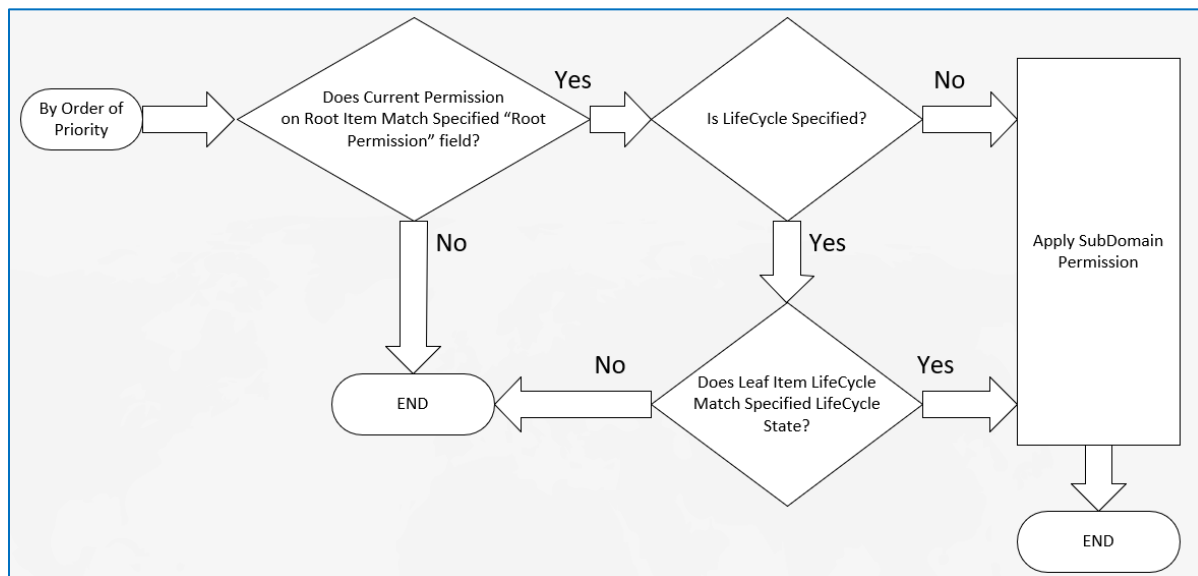


Figure 5.

For example, consider subdomain rules described in Table 3.

Table 2: Subdomain rules

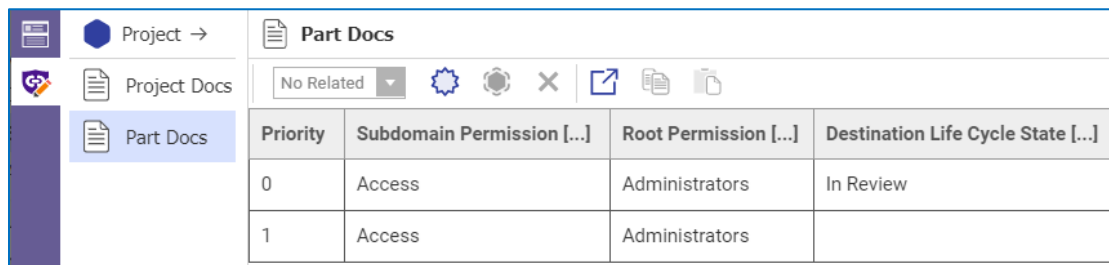
Subdomain Permission	Root Permission	Destination Life Cycle State
Preliminary_Document	Preliminary_Project	
ReadOnly_Document	Released_Project	Released
DiscoverOnly_Document	Released_Project	Superseded

If these Rules were set on the **Project->Document** subdomain it would imply the following:

- If the Permission_ID on the Root (Project) is the ID of 'Preliminary_Project' then apply 'Preliminary_Document' permission to the Leaf (Document).
- If the Permission_ID on the Root (Project) is the ID of 'Released_Project' AND the LifeCycle State of the Leaf (Document) is 'Released' then apply 'ReadOnly_Document' permission to the Leaf (Document).
- If the Permission_ID on the Root (Project) is the ID of 'Released_Project' AND the LifeCycle State of the Leaf (Document) is 'Superseded' then apply 'DiscoverOnly_Document' permission to the Leaf (Document).

The **Priority** Field works as follows:

- No two rules may have the same Root Permission and LifeCycle State. This will issue a validation error when saving DAC Definition.
- If the Life Cycle State is left blank (implying 'any') then the Priority value must be a higher positive integer than any specified Life Cycle State.



Priority	Subdomain Permission [...]	Root Permission [...]	Destination Life Cycle State [...]
0	Access	Administrators	In Review
1	Access	Administrators	

Figure 6.

1.2.1.1 Example Scenario: Using a Subdomain Rule for Project Domain Access Control

The following is an example scenario of how a Subdomain Rule might be used:

- The Document Itemtype (outside of DAC) is configured with default "Discovery Only" access for All Employees.
 - Without DAC, MAC or Lifecycle driven permissions, this is the only access granted to employees.
 - Non-employees would have no access.
- The Project Itemtype has a RelationshipType 'Project Document' with a related Document ItemType.
 - In DAC, a Query Definition is used to identify this relationship as a DAC Subdomain.
- The Access Policy has the following business requirements:
 - All Project Team members receive "Get" access to Documents when added to the Domain and if the Root Permissions allow it. Access is then elevated for the Team. Users with the Administrator Role in the Project Team also getting "Can Change Access" rights in this case.
- A Rule is defined as follows:
 - Root Permission = 'Released Project Permission.'
 - Destination Life Cycle State (blank).
 - Subdomain Permission = 'Project Domain Member Access':
 - Team Member: Get access.
 - Team Administrator: Get + Can Change Access.

- This Rule is added to the Subdomain and the DAC Policy is activated.
- A Document is added to the Project Relationship structure (Project Domain):
 - Result: Team members can now open the form and view the Document.
- Team members with the Administrator Role also have 'Can Change' access.

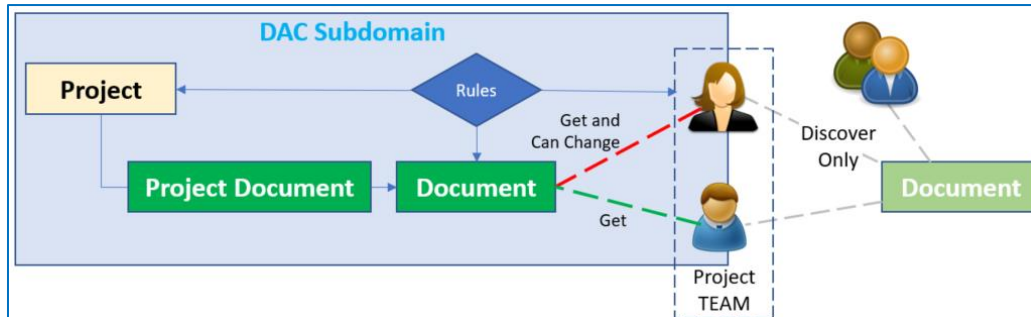


Figure 7.

Subsequently rules can be added for different Root Permissions or LifeCycle States on the destination (Leaf) item, additional Team members, etc.

1.2.2 Mapping Team Roles from the Root Item Team

If there is a Team assigned to the Root item, and Roles are used in the Subdomain Permission (the permission that is assigned by the Rule calculation) then the Roles must match, otherwise there will not be a dynamic assignment by Role when the permission is applied. See section 2.5 “Creating Subdomain Permissions” for more information.

1.3 Domain Access Control (DAC) vs. RBAC and MAC

DAC can either grant or elevate access levels for users in the following way:

- If the User (Role, Team Member) has no access via standard permissions, DAC can be configured to provide or raise access levels using DAC Rules.
- MAC can conditionally deny access even if access is granted by RBAC and DAC.

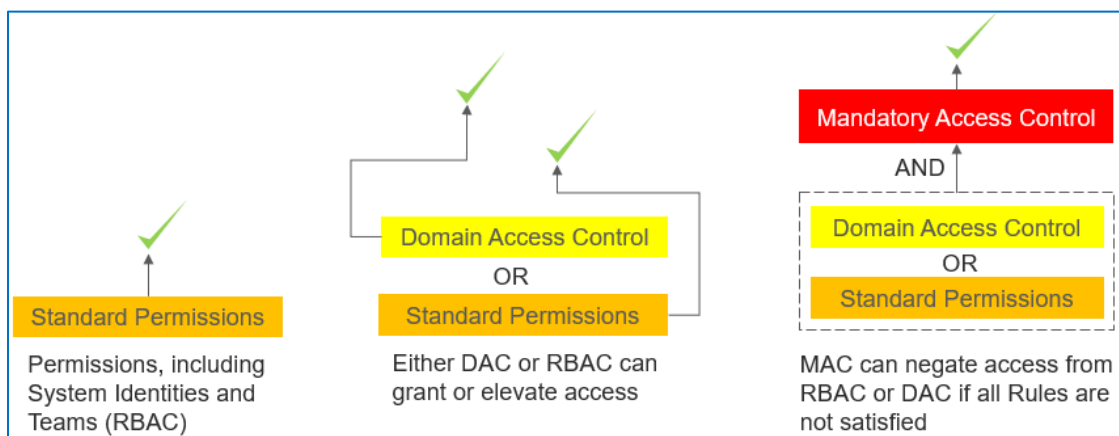


Figure 8.

- RBAC assigns permissions directly on the item, which can only change with the lifecycle state (or authorized editing).
- DAC assigns permissions to leaf items under a subdomain if the associated rules are satisfied. The combination of RBAC and DAC Policy is “least restrictive”, meaning either one can grant permission to the current user.
- MAC imposes strict conditions on a user’s access to items by evaluating attributions on user vs. item in conditional (Boolean) expressions. If conditions are not TRUE, then access is revoked even if granted by RBAC/DAC Policies.

1.4 DAC Data Model Considerations

When configuring a DAC policy, it is useful to understand the DAC components particularly because it is configured “bottom-up” by administrators as you will see in Section 2.

DAC is far from monolithic. The primary ItemType in DAC is the **DAC Definition** however it depends heavily on the **Derived Relationship** ItemType and **Derived Relationship Family** which in turn depends on the **Query Definition**. All of these must be configured before the DAC Definition can be completed.

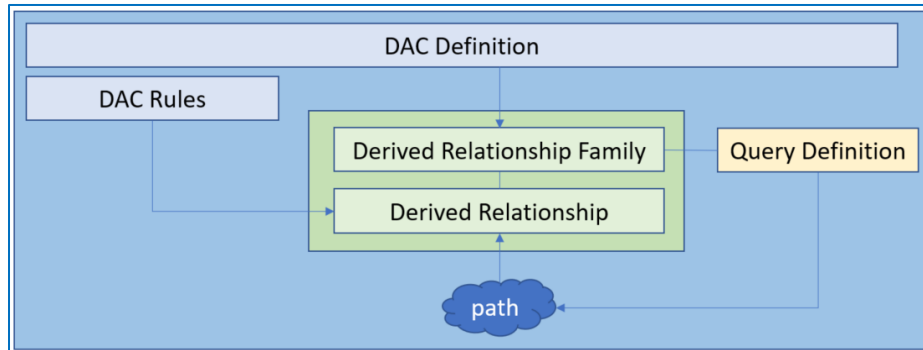


Figure 9.

The Query Definition selects paths and endpoints from the physical relationship structure for use as Derived Relationships (DR) in a Derived Relationship Family (DRF). The DRF tracks changes to controlled items on behalf of DAC.

When defining a DAC Policy first create the Query Definition, use it to complete the DRF and its DRs, and then define rules for the DAC Definition.

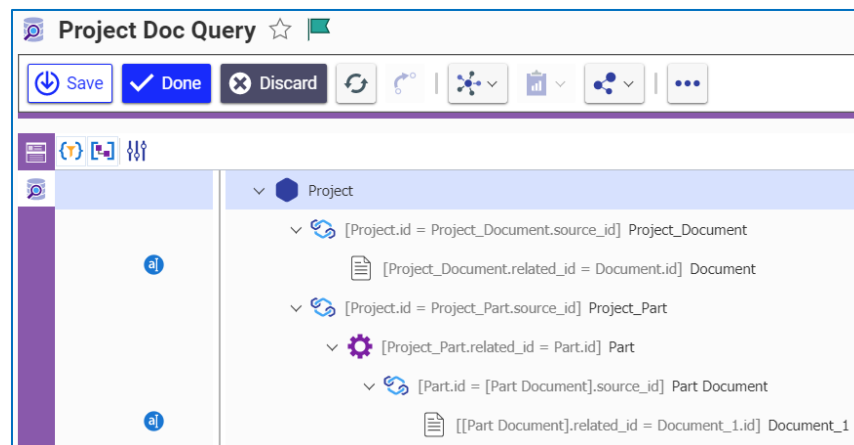


Figure 10.

When the Query Definition is linked to the Derived Relationship Family item, Leaf items can be selected (Root is implied) to define Derived Relationships.

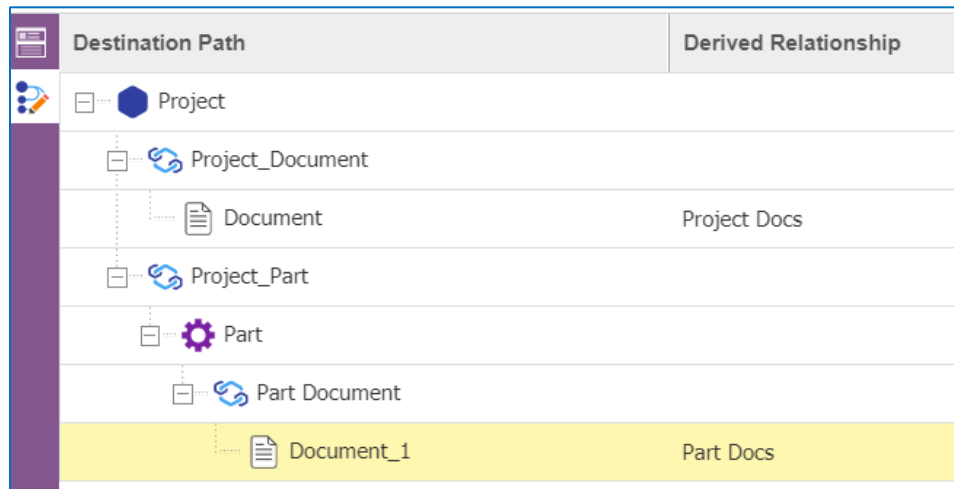


Figure 11.

Finally, the DAC Definition links to the DRF and you can assign the Subdomain Rules to each DR.

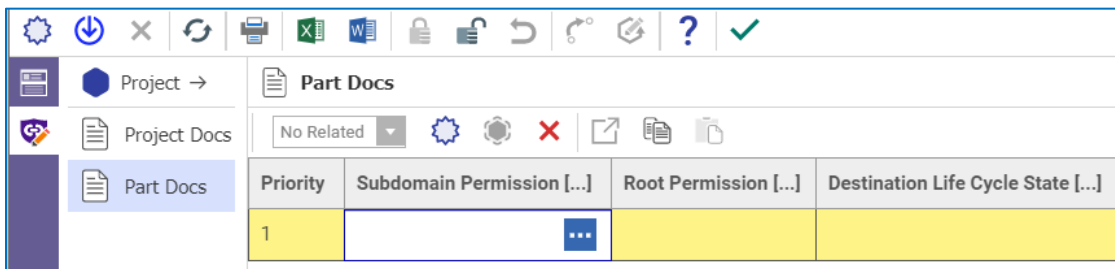


Figure 12.

Section 2 Creating and Activating a DAC Domain Policy walks through each step.

1.4.1 Tracking Changes to Access Controlled Items

When a DRF is first activated, an ItemType table called **unidr_Relationship** is populated with query results for every instance of Root->Leaf defined by the Derived Relationships in the family. This may take some time for large databases.

Records in this table contain references to all instances of Root and Leaf items in DAC Subdomains. After first-time initialization, the table is maintained automatically when events (add, delete, modify, etc.) occur against member items in DAC Subdomains. In this manner DAC maintains the integrity of Access Control for all item instances in the policy.

The **unidr_Relationship** table includes these columns:

Derived Relationship (below...)	Departure (root)	Destination (leaf)
(ID)	(ID)	(ID)
etc	etc	etc

Figure 13.

- Derived Relationship [...]
 - Pointer to DR under DRF linked to DAC Definition containing DAC Rules.
- Departure [...]
 - ID of the Root item instance of the Subdomain (e.g. Project X).
- Destination [...]
 - ID of Leaf item instance (e.g. Document D-123).

As of 12.0 SP5, DRF event handlers have been made configurable / customizable for those who need to execute business logic when changes to item memberships in DAC Subdomains occur.

1.4.2 Customizing DRF Event Handlers

Earlier we discussed the scenario where Project Team “X” gains access to an item added to the Project from another Project under DAC Policy. In some cases, business requirements prohibit the increase of access rights due to inclusion into a Domain.

For instance, a Part belonging to Project Y might be re-used by adding it to Project X. The Project X Team may suddenly gain Update rights to the Part shared with Project Y. The intent may have been to allow reuse but not open the Part from Project Y for modification by the other Project Team X.

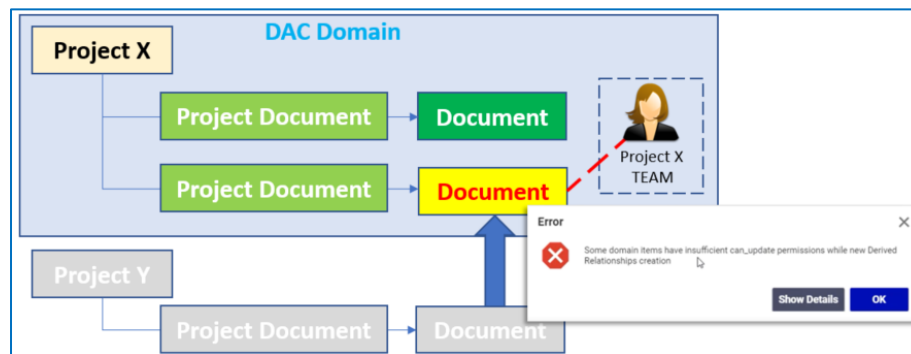


Figure 14.

With 12.0 SP5, DAC Domains provide a hidden ‘Events Handlers’ tab under DRF so that Administrators can either specify which Access Levels change(s) are prohibited, enable or disable such checks, or add custom logic as needed.

The Administrator now has the choice to:

- Configure the DAC Definition for different Access Types (Update, Delete, etc).
- Use custom event handler method in place of the default or ,
- Add a new Event method to execute in sequence with the default Event Handler from Aras.

Configuring/customizing DRF Event Handlers is an advanced feature. Details on DRF Event Handlers is provided in the Appendix.

1.5 Terminology

The following terms are used in this guide to describe DAC:

General Terms:

- **Access Controlled Items** – Items under DAC policy control, specifically the related items under a relationship structure that is governed by a DAC policy.
- **DAC Domain** – A logical grouping consisting of all related items sharing the same Root item whose access control policies are defined by a DAC policy.
- **DAC Subdomain** - A single path inclusive of root and leaf items in a DAC Domain.
- **Subdomain AC Policy** - the set of Access Rules that govern permissions for a Leaf item within a subdomain (path).
- **Subdomain AC Rule** – Determines which permission to grant to a Controlled Item using the configured criteria.
- **Root Item** – Top-most node of a Relationship Structure, Context Item of the Query Definition used to define Subdomains.
- **Leaf Item** – ‘End’ item in any path of a Relationship structure. The item whose access is controlled by DAC Policy (Access Controlled item)
- **Path** – A segment from a Relationship Structure connecting a Root Item to a Leaf Item

DAC ItemTypes:

- **DAC Definition** – An ItemType that defines a DAC policy against a domain using a Query Definition, Derived Relationship Family and DAC Subdomains. A Container for Subdomain Rules.
- **Query Definition** – A standard Aras Query Definition, used to specify paths from the controlled Relationship structure against which DAC will be applied.
- **Derived Relationship (DR)** - A Named ItemType that identifies start and endpoints as a path within a relationship structure. The start, or Departure ItemType is the root item and the Destination ItemType is the leaf item. In the DAC context it is analogous to a subdomain.
- **Derived Relationship Family (DRF)** – An ItemType that contains a Query Definition and the associated group of Derived Relationship items defined by that Query. Derived Relationships are analogous to DAC subdomains (start/end points of a path within the overall structure).
- **unidr_Relationship** – Persisted table used by DAC to track subdomains and membership of leaf items therein.

2 Creating and Activating a DAC Domain Policy

The procedure for creating and activating a DAC policy is described in detail in the following sections. As described earlier it is a bottom-up process.

- **Creating a Query Definition:**
The Query Definition specifies the paths from the Root item through one or more Relationships to Leaf Items. It is accessed from the TOC folder *Administration/Configuration*.
- **Creating a Derived Relationship Family (DRF):**
A Derived Relationship Family is an ItemType that contains a Query Definition and one or more relationship paths derived from this query known as Derived Relationships. It can also be found in the TOC folder *Administration/Configuration*.
- **Defining Derived Relationships:**
Derived Relationships are related items on the “Derived Relationships” tab on the DRF. Each has a Name identifier and start/end points of the DAC Subdomain in the columns “Departure Item Type [...]” and “Destination Item Type [...]”.
- **Persisting DAC Access Control Table (*unidr_Relationships* table):**
Once a Derived Relationship Family and its subdomain definitions are completed, the ‘unidr_Relationships’ table is populated with data to track all instances of the specified subdomain in the database. This may take quite a bit of time for large databases.
- **Creating a DAC Definition (Policy):**
Once the Derived Relationship Family is persisted, we create a DAC Definition that applies DAC Access Rules for each DRF subdomain conditionally based on root item permissions and leaf item life cycle states.

An active DAC Definition automatically creates DAC Domains for all instances of the root item. Access is based on the Team attached to the root item and governed by the permissions set in the DAC Definition. DAC access is calculated based on the table of *unidr_Relationship* items which are automatically synchronized with the results of the Derived Relationship Family’s Query Definition.

2.1 Prerequisites

Before using Domain Access Control, you must make sure that the Agent Service is running. Ensure that the Agent Service Conversion Processing Cycles are configured in the **conversion.config** file, with the appropriate DB and User filled in:

```
<ProcessingCycles>
  <cycle DB="{db}" User="{user}" Interval="20000" MaxBatch="5"/>
</ProcessingCycles>
```

Restart the Agent Service after making changes to the config file.

The Agent Service also executes Conversion Tasks generated during the persisting process to create the initial *unidr_Relationship* items.

Note: Examples in this document use the Project ItemType. This ItemType is now an Aras Package that is installed separately. If you do not have this add-on installed, use Aras Update to install it before proceeding.

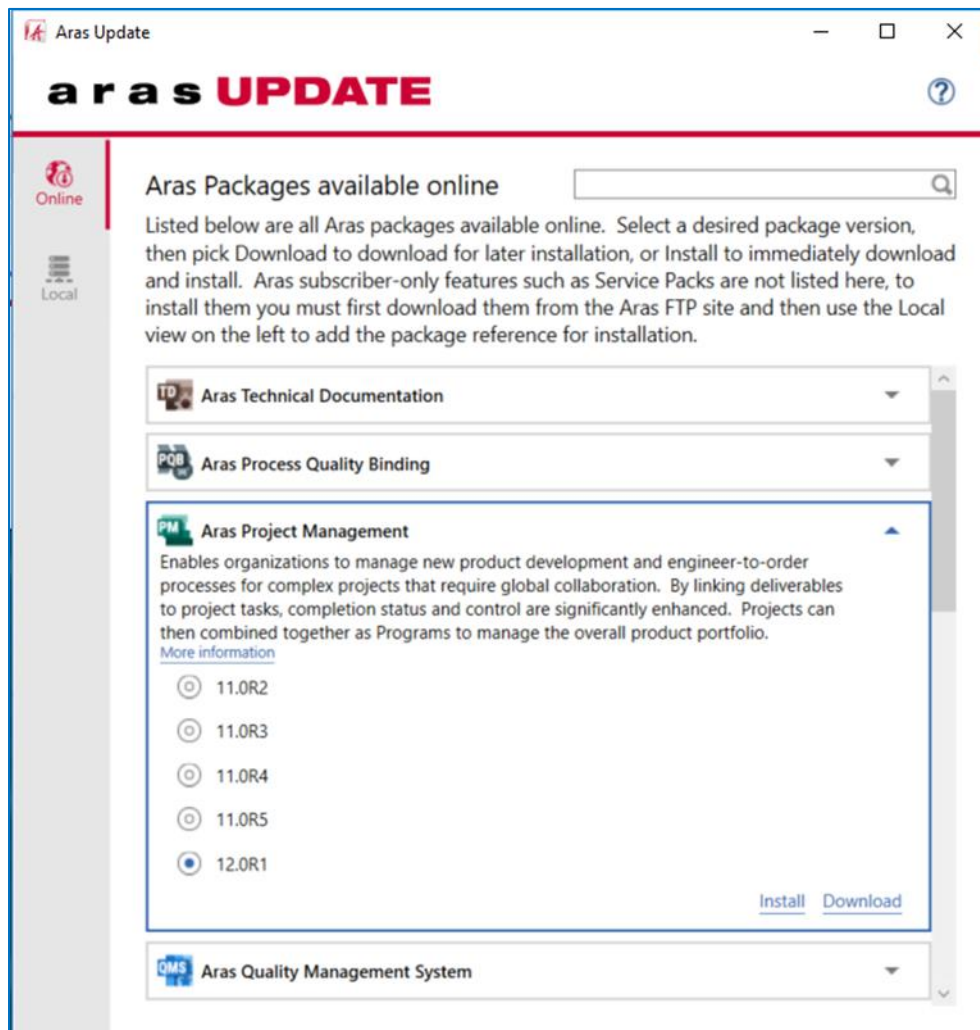


Figure 15.

2.2 Creating a Query Definition

You need to create a Query Definition in order to create a Derived Relationship (DR) Family. The DR Family Definition maps DR Family elements and non-root Query Items contained in the Query Definition, which are in turn used to create DAC Subdomain Policies. For information on how to create a Query Definition, refer to the *Query Builder Guide*. In this guide we will use the following example Query Definition:

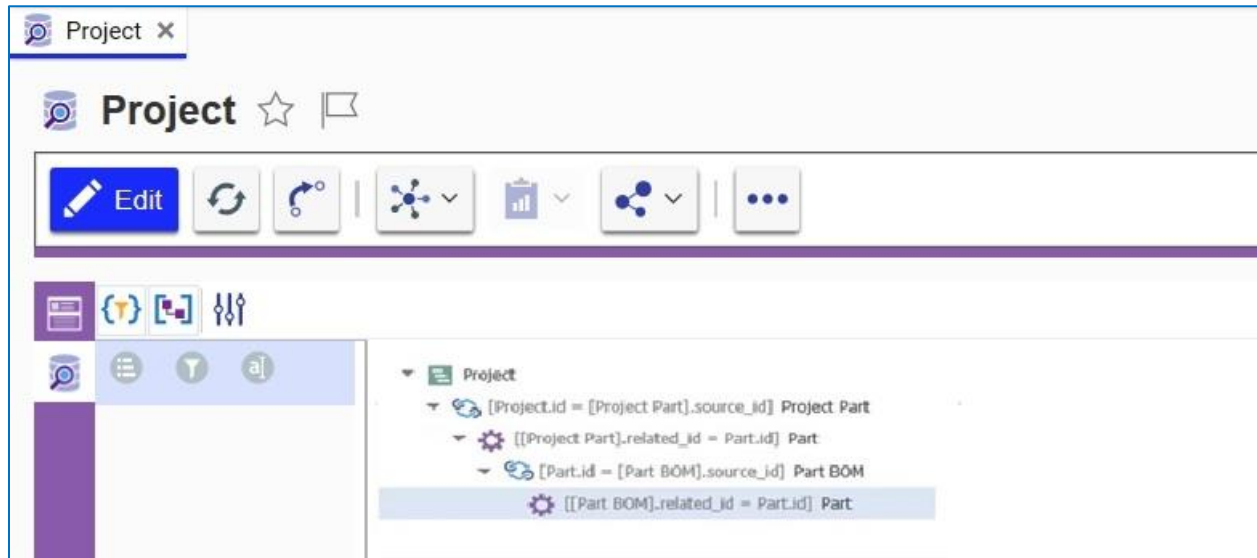


Figure 16.

Note: The root Query Item (Project in this example) should not be reused in a Query Definition that will be used by a DR Family.

2.3 Derived Relationship Families

A Derived Relationship (DR) Family describes the derived relationships between Aras Innovator items. Each Derived Relationship in a DR Family has its own destination ItemType and name. An individual DR is referred to as a DR Family Element. All DR Family elements share the same departure ItemType as determined by the Query Definition.

2.3.1 Creating a Derived Relationship Family

Use the following procedure:

1. In the Navigation Panel, click **Administration>Configuration>Derived Relationship Families**.
The following menu appears:

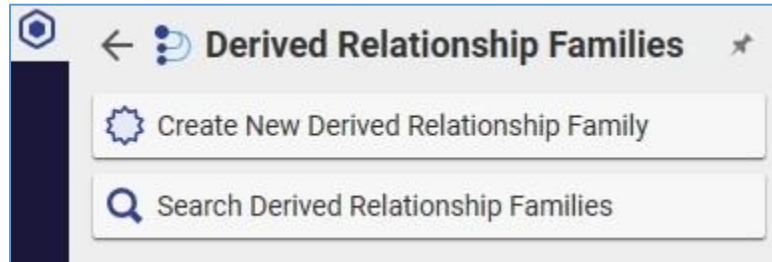


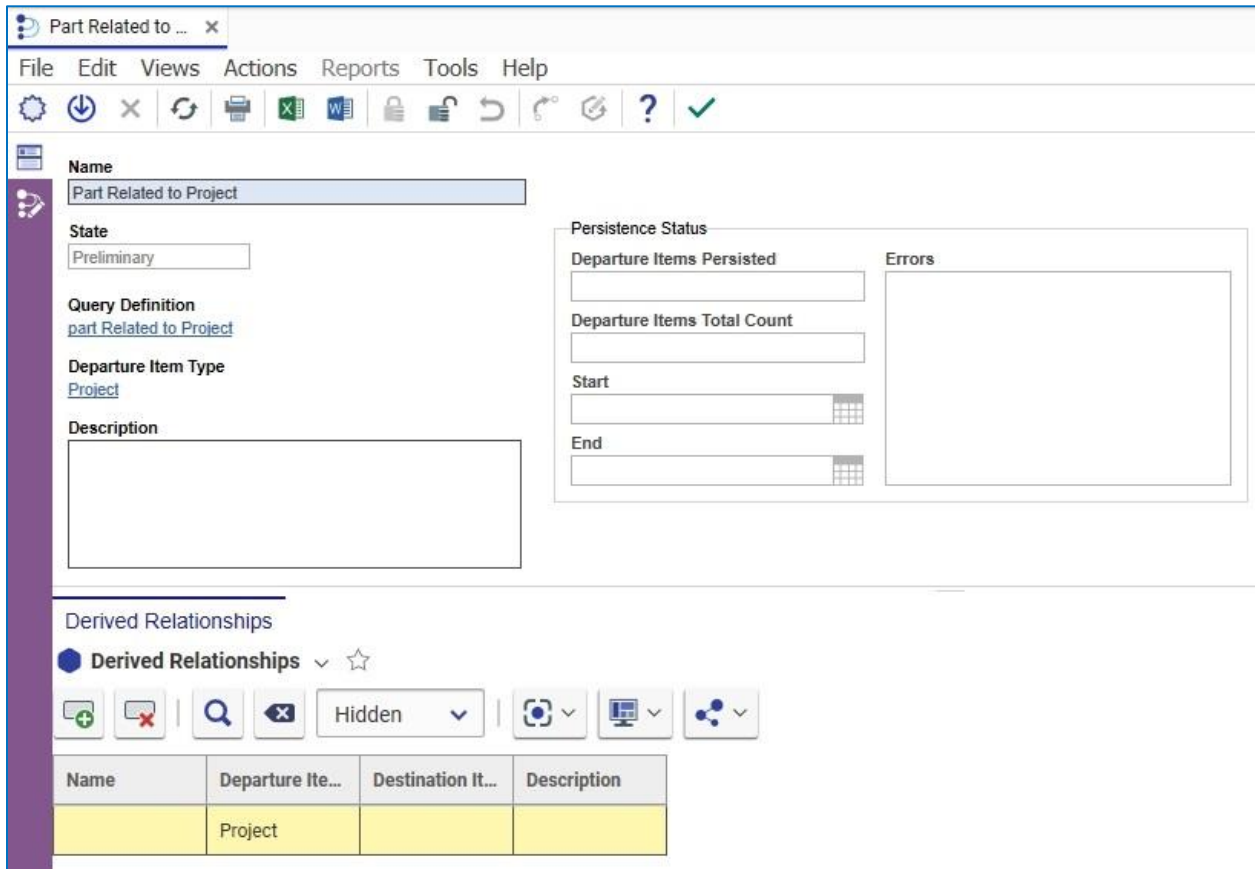
Figure 17.

2. Click **Create New Derived Relationship Family**. A screen like the following appears:

Figure 18.

3. Enter a name for the derived relationship in the **Name** field.
4. Click the ellipses in the **Query Definition** field and select the Query Definition from the dropdown list. Once you click save you may not change the assigned Query Definition.
5. Save the Derived Relationship Family. Its state is now Preliminary. The Query Definition is claimed and the Departure ItemType is updated based on the Query Definition.

6. Click the **New Derived Relationship** icon  in the Derived Relationships section to create a Derived Relationship.



Part Related to ...

File Edit Views Actions Reports Tools Help

Name
Part Related to Project

State
Preliminary

Query Definition
part Related to Project

Departure Item Type
Project

Description

Persistence Status

Departure Items Persisted

Departure Items Total Count

Start

End

Errors

Derived Relationships

Derived Relationships

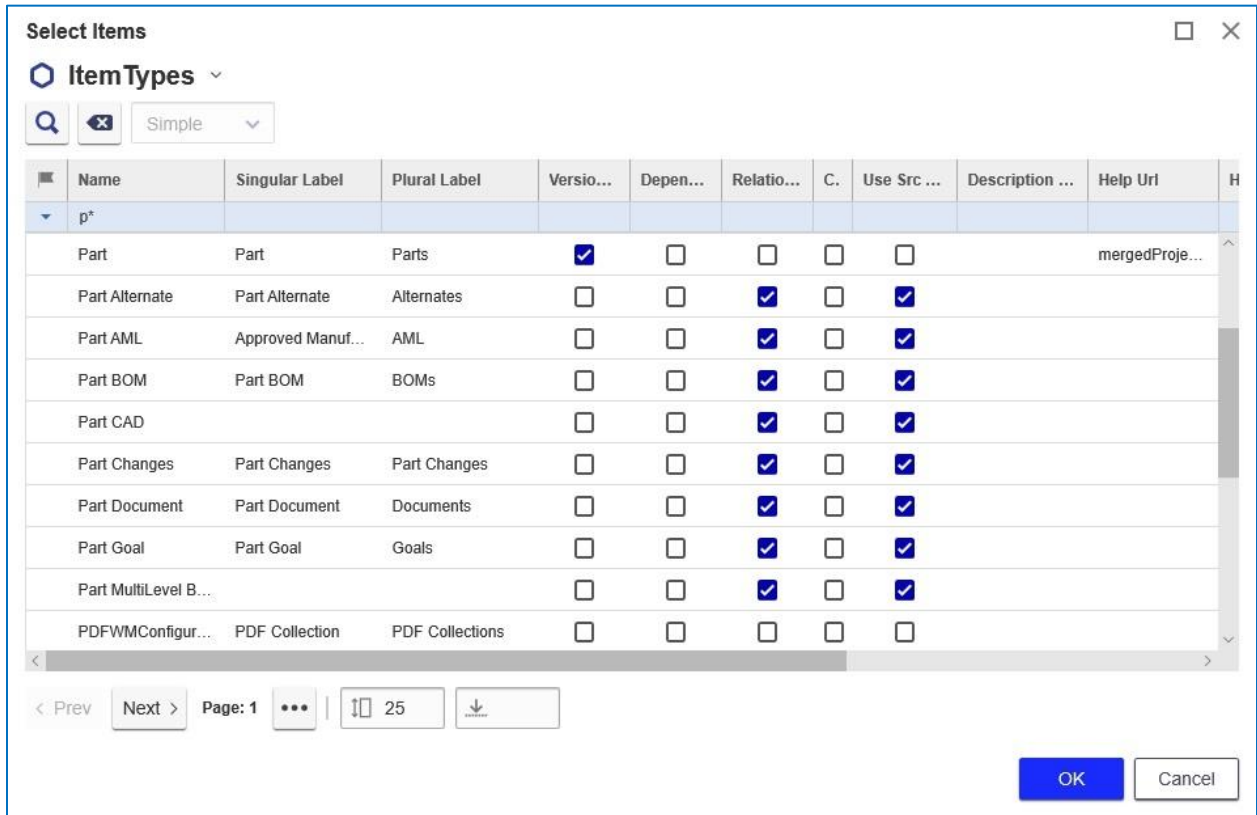
Name	Departure It...	Destination It...	Description
	Project		

Figure 19.

The Root Item Type name (in this case Project) appears automatically. It is not editable.

7. Enter the Derived Relationship name in the **Name** field of the Derived Relationship tab.

8. Click the **Destination Item Type** field. The ItemType search dialog box appears.



The 'Select Items' dialog box displays a list of ItemTypes. The 'Part' item type is selected. The dialog includes a search bar, a filter dropdown, and a table of ItemTypes.

Name	Singular Label	Plural Label	Versio...	Depen...	Relatio...	C.	Use Src ...	Description ...	Help Url
Part	Part	Parts	☒	☐	☐	☐	☐		mergedProje...
Part Alternate	Part Alternate	Alternates	☐	☐	☒	☐	☒		
Part AML	Approved Manuf...	AML	☐	☐	☒	☐	☒		
Part BOM	Part BOM	BOMs	☐	☐	☒	☐	☒		
Part CAD			☐	☐	☒	☐	☒		
Part Changes	Part Changes	Part Changes	☐	☐	☒	☐	☒		
Part Document	Part Document	Documents	☐	☐	☒	☐	☒		
Part Goal	Part Goal	Goals	☐	☐	☒	☐	☒		
Part MultiLevel B...			☐	☐	☒	☐	☒		
PDFWMConfigur...	PDF Collection	PDF Collections	☐	☐	☐	☐	☐		

Navigation controls: < Prev, Next >, Page: 1, 25, Download.

Buttons: OK, Cancel.

Figure 20.

The dialog displays the ItemTypes associated with the Query Definition.

9. Select the Destination ItemType. In this example we select **Part**.

10. Repeat this process and add all necessary Derived Relationships.

Part Related to ...

File Edit Views Actions Reports Tools Help

Name
Part Related to Project

State
Preliminary

Query Definition
part Related to Project

Departure Item Type
Project

Description

Persistence Status

Departure Items Persisted

Departure Items Total Count

Start

End

Errors

Derived Relationships

Derived Relationships

Name	Departure Item Type [...]	Destination Item Type [...]	Description
Part Related to Project	Project		

Figure 21.

11. Click the **Query Definition** icon in the sidebar to view the query definition.

Derived Relationship ...

File Edit Views Search Actions Reports Tools Help

Destination Path

Derived Relationship

Project

Project Part

Part

Part BOM

Part_1

Figure 22.

12. Click on a **Derived Relationship** cell to map the Query Item to the Derived Relationship that you created on the previous screen.

13. Select the corresponding DR from the dropdown list.

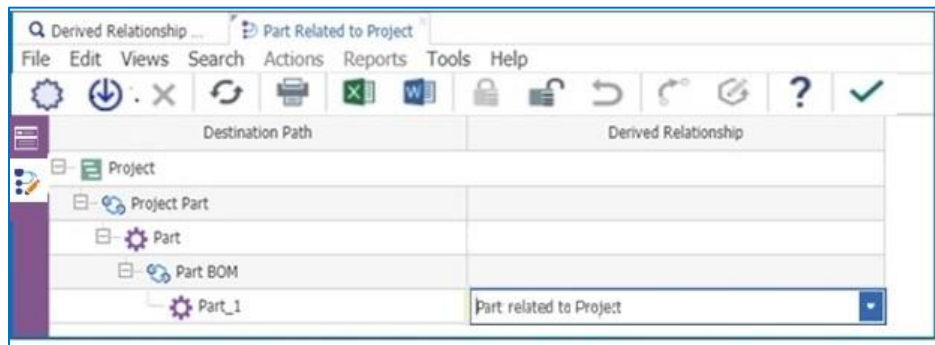


Figure 23.

Note: You can only specify Derived Relationships which have been created on the previous screen.

14. Once all Derived Relationships are mapped click **save** and then **unclaim** the DR Family.
15. From the Actions Menu, select the Activate action to promote the DR Family to the Active state. The status of the Derived Relationship changes to **Persisting** and the initial persistence of the DR Family begins. Conversion Tasks are created which will persist the DRs for each QD root item in the database. By default, 10 Departure Items (Can be configured by modifying the dr_DepartureItemsPerConvTask system variable) are processed per Conversion Task. When the last Conversion Task is finished, the DR Family is activated

Note: During this process you should not change data in any way that may affect the outcome of the DR persistence operation. The initial DR persistence process may take some time depending on the size of the database.

Information relevant to the Persistence process can be viewed in the Persistence section of the Derived Relationship Family Form. Departure Items Persisted, % displays the percent of Departure Items that have been persisted. Departure Items Total Count provides the total number of Departure Items in the system. If there are errors during the persistence process, the Errors box is populated with information pertaining to the errors.

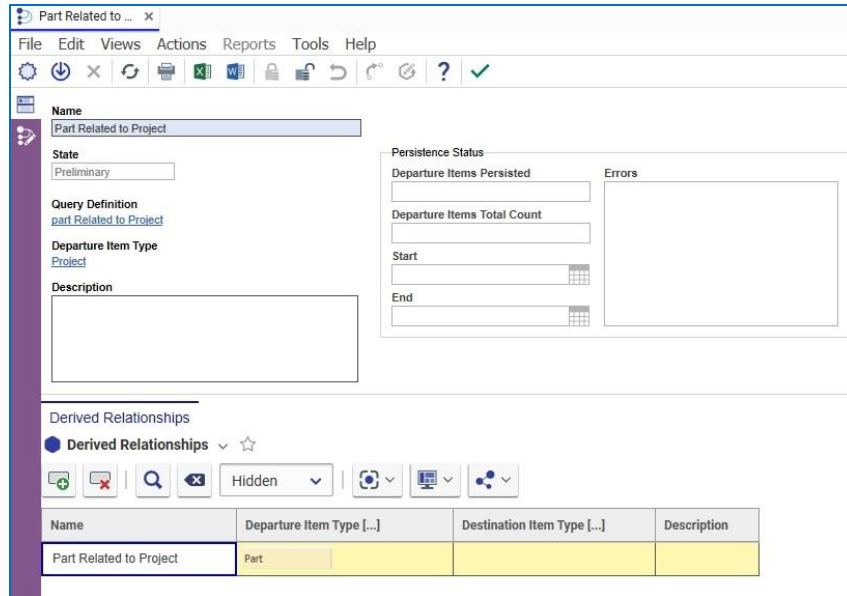


Figure 24.

If all the Conversion Tasks succeed, the DR Family is automatically promoted to **Active**. You cannot edit the Derived Relationship Family once it is Active.

2.3.2 Active Derived Relationship Families

Data changes in the database may create a need to recalculate the DR data to keep it up to date. This is done through observers that react to each change made to the items associated with ItemTypes participating in the DR Family. When an observer catches a change, it updates (re-calculates) the persisted DR as needed. An update impacting one DR of a DR Family may necessitate an update of other DRs within the Family.

2.3.3 Deactivating a Derived Relationship Family

You can only deactivate a DR Family if it is not in active use. To deactivate a DR Family, select the Deactivate action from the Actions menu. Upon deactivation of a DR Family all the persisted Family DRs are deleted. The observers that have been activated for DR Family “participating ItemTypes” are deactivated. Deactivated DR Family can be deactivated and then activated again.

Part Related to ... x

File Edit Views Actions Reports Tools Help

Name
Part Related to Project

State
Preliminary

Query Definition
part Related to Project ...

Departure Item Type
Project

Description

Persistence Status

Departure Items Persisted

Departure Items Total Count

Start

End

Errors

Derived Relationships

Derived Relationships v ☆

Hidden v

Name	Departure Item Type [...]	Destination Item Type [...]	Description
Part Related to Project	Project		

Figure 25.

2.3.4 Derived Relationship Family Remediation

DRF remediation is a process of resolving denormalized data when the denormalized data gets out of sync with the primary data. This process can be run either manually from the DRF action menu in Aras Innovator, or automatically at certain times of the day. Both manual and automatic DRF remediation assumes that the primary DRF data is not modified by end users during the duration of the DRF remediation process. (If this assumption is incorrect, DRF remediation process cannot be run.) Both manual and automatic remediation record the result of the undertaken remedial action in the Aras Innovator log files, and in the Conversion Tasks that are used to drive the DRF remediation process.

How to do manual DRF Remediation?

Admin can do the Remediation for a specific DRF via  and Remediate action any time when it's needed:

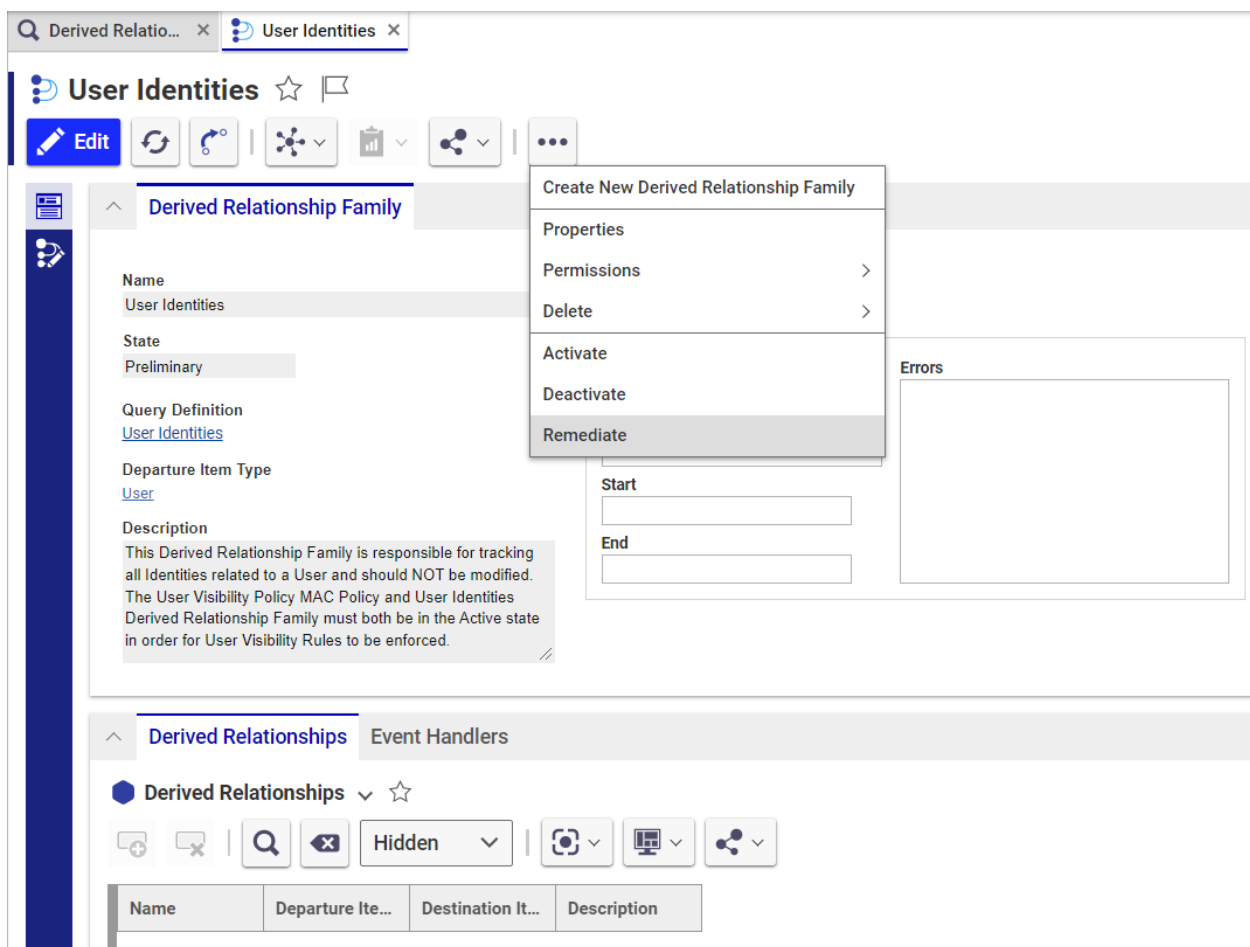


Figure 26.

How to set up automatic DRF Remediation

This functionality enables an Admin to configure the system to automatically recalculate and resolve issues with the Derived Relationship Families on a scheduled basis.

Administrators should create a daemon process that will periodically check correctness of the system DRFs and perform required repairs when needed:

The Agent Service should be set up as a prerequisite to this process (details are provided in 2.1 Prerequisites)

Step 1 – Activate target DRF, that you need to Remediate on a regular (automatic) basis

Step 2 – Install scheduler: run InstallService.bat from Scheduler folder on CD Image. For more details you can also refer to Aras Innovator 33 - Scheduler Service.pdf of the latest released build.

Step 3 – Create the method with the following code where target DRF ID and departure_id is replaced with the correct values.

```
var schedServ = this.newItem("dr_RelationshipFamily", "dr_RemediateDRs");
schedServ.setID("target DRF id");
schedServ.setProperty("departure_ids", "");
schedServ.setProperty("is_auto_remediation", "1");
return schedServ.apply();
```

Step 4 – Admin can change **Innovator/Server/appsettings.json** config file to see logs with the following information "MinimumLevel": "Fatal", -> "MinimumLevel": "Information".

By default, the automatic remediation process logs the fixes only, that are made in UNIDRs data. If User wants to log all UNIDRs values (even those ones that stayed the same) then dr_RemediationLoggingLevel Innovator Variable should be set to 'verbose'.

Step 5 – Information on the results of DRF remediation can be found in two places:

Innovator log files

Innovator\Server\logs\InnovatorServer_YYYYMMDD.log
(Aras.DerivedRelationships.Loggers.RemediationProcessLogger), e.g.

```
2021-10-07T17:10:57.757 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Application | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.757 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.759 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Application | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.759 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.875 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.875 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.875 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.875 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.875 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.876 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.877 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.885 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Metadata | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.939 [INFO] Aras.DerivedRelationships.Loggers.RemediationProcessLogger | Universal Derived Relationships were removed by automatic remediation process | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.969 [INFO] Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor | Executing ObjectResult, writing value of type 'System.Xml.XmlDocument'. | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.970 [INFO] Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker | Executed action 'Aras.Server.Controllers.InnovatorServerController.ProcessInnovatorServerRequest (Aras.Server)' in 1088.463ms | 5e
2021-10-07T17:10:57.970 [INFO] Serilog.AspNetCore.RequestLoggingMiddleware | HTTP POST /s14-0-sp2/Server/InnovatorServer.aspx responded 200 in 1089.0704 ms | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:10:57.970 [INFO] Microsoft.AspNetCore.Hosting.Diagnostics | Request finished in 1091.038ms 200 application/xml | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:11:02.355 [INFO] Microsoft.AspNetCore.Hosting.Diagnostics | Request starting HTTP/1.1 POST http://localhost/s14-0-sp2/Server/InnovatorServer.aspx text/xml; charset=UTF-8 448 | SessionId:
2021-10-07T17:11:02.355 [INFO] Microsoft.AspNetCore.Cors.Infrastructure.CorsService | CORS policy execution failed. | SessionId:
2021-10-07T17:11:02.355 [INFO] Microsoft.AspNetCore.Cors.Infrastructure.CorsService | Request origin http://localhost does not have permission to access the resource. | SessionId:
2021-10-07T17:11:02.356 [INFO] Microsoft.AspNetCore.Mvc.Infrastructure.ControllerActionInvoker | Route matched with {action = "ProcessInnovatorServerRequest", controller = "InnovatorServer"}. Executing controller action with s
2021-10-07T17:11:02.361 [INFO] Aras.Server.Core.CacheManager | Cache synchronization started for Application | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
2021-10-07T17:11:02.364 [INFO] Microsoft.AspNetCore.Mvc.Infrastructure.ObjectResultExecutor | Executing ObjectResult, writing value of type 'System.Xml.XmlDocument'. | SessionId: f2407149-5418-14c8-le03-ffecb2a99fe
```

Figure 27.

Conversion Tasks used for remediation processing

- Open Conversion task that was created for this Remediation process and see the statistics for it in User data property, e.g.

Conversion Task

Status: Succeeded

Rule: [dr_DerivedRelationshipGeneratorRule](#)

File Type: File

Started On: 5/9/2022 5:12:36 PM

Finished On: 5/9/2022 5:12:37 PM

User Data

```
<AML>
  <Item type="dr_RelationshipFamily" id="D630DE7BBB804D1BA296344A127E2D56">
    <departure_ids>50B53A6CC84D3E4ABDCAFE0157C49AC,50E9C7F4C615BAA001AE7A3A2B1ECDE3,E173C0E2A5FFEE6DBBBF1077B61D46AEC,864E72AECDDDD5FAD3FF032582C7E3CD,E4AD23A7FA6CD0DE4ADB78F7B6332A07,B65BFFDF62772EEF4EBA65FD5A0DFA6A,D2B14E33EE5D481E9BD6F7D4FA48095A,D5DFF57BE221F0F72CEAF9B0A263FB3C,E6ED0DD88F324AF48B85C6A215546779,E84E8EB9DDAD7B5E8ED42507AB723C3C</departure_ids>
    <remediation_created_on>2022-05-09T14:12:28</remediation_created_on>
    <added_count>3</added_count>
    <removed_count>2</removed_count>
  </Item>
</AML>
```

Figure 28.

- <is_auto_remediation> - flag equals to '1' that indicates the automatic run of remediation process, otherwise it does not exist
- <added_count> - quantity of records that was added during Conversion Task(s) execution
- <removed_count> - quantity of records that was deleted during Conversion Task(s) execution

2.4 DAC Definitions

When you create a DAC definition, you must specify the DAC Definition name and the Derived Relationship Family. Once you save this information, you cannot modify the definition to include a different DR Family. The DR Family defines the root ItemType of the DAC Definition.

You need to create a Subdomain policy for each of the DR Family elements and the associated subdomains. The Subdomain name is the same as the corresponding Derived Family element. The ItemType for the subdomain items is the Derived Family element destination ItemType.

You may then define a DAC Subdomain policy for each of the subdomains included in the DAC Domain definition. Once you create a DAC definition, you need to activate it. The following diagram shows the DAC Definition Lifecycle.

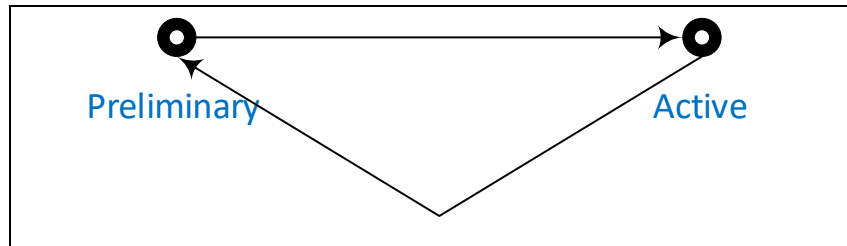


Figure 29.

When a DAC Definition is in the Preliminary state, it means that it is being created/edited. The appropriate DAC domains have not been created yet. In this state the DAC domain has no effect on access control.

Once you create and validate a DAC Definition, the Administrator activates it. The corresponding DAC Domains are created dynamically using new root items. The Access Control logic takes this information into account when determining an access request. The procedure described in the following section is an example of creating a DAC Definition.

2.5 Creating Subdomain Permissions

Use the following procedure to create the appropriate access permissions to be used for DAC Subdomain Policies.

1. In the Navigation Pane, click **Administration>Permissions**. The following menu appears:

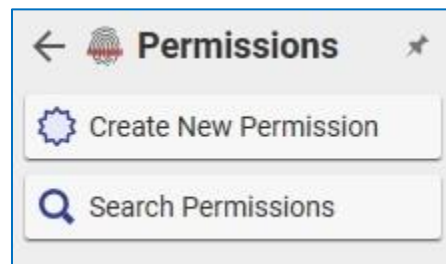


Figure 30.

- Click **Create New Permission**. The following screen appears.

Permission 1 x

Permission 1

Save Done Delete

Permission

Name

Access

Identities

Hidden

Name	Get	Upd...	Dele...	Can Discover	Show Permis...	Can change a...
Team Manager	☐	☐	☐	☐	☐	☐
Team Member	☐	☐	☐	☐	☐	☐

Figure 31.

- Enter the appropriate permission name in the Name field.

- Click the **Add Identities** icon  on the Access tab to specify permissions for existing identities. Click the **New Identities** icon  to create an identity to associate with the Permission.

Part Related to ... x

Part Related to Project ☆

Edit

Permission

Name

Part Related to Project

Access

Identities

Hidden

Name	Get	Upd...	Dele...	Can Discover	Show Permis...	Can change a...
Team Manager	☐	☐	☐	☐	☐	☐
Team Member	☐	☐	☐	☐	☐	☐

Figure 32.

Note: If a Team is associated with the Domain Root ItemType (Project in our example), roles in the Subdomain permission need to match those in the team in order to take effect. Refer to the [Creating Teams](#) section in the Appendix for more information.

- Click  to save and unclaim the Permission item.

2.6 Creating a DAC Definition

- In the Navigation pane, click **Administration>Access Control>DAC Definitions** from the TOC. The following menu appears:

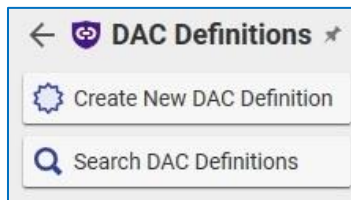


Figure 33.

- Click **Create New DAC Definition**. The DAC Definition window appears:

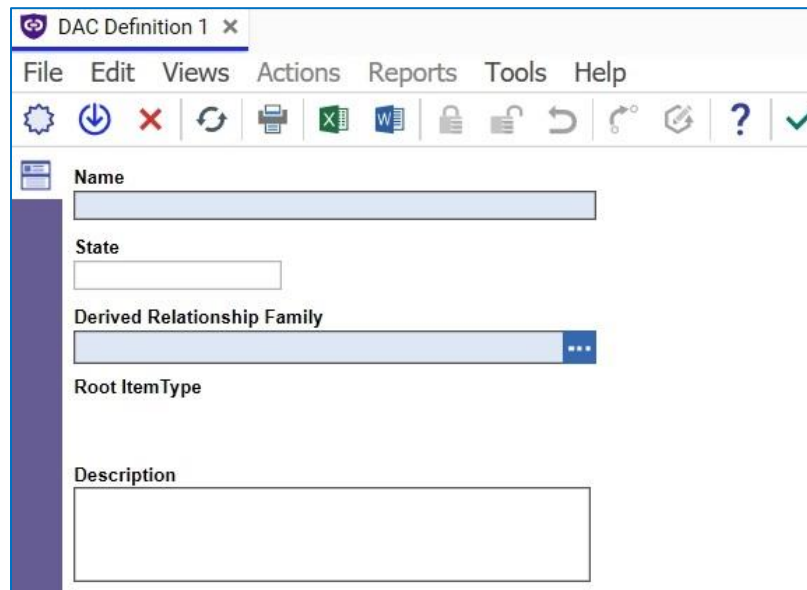


Figure 34.

- Enter a name for the DAC Definition in the **Name** field.

- Click the ellipses in the **Derived Relationship Family** field to select the appropriate DR Family.

The screenshot shows a software window titled "DAC Definition 2". It has a menu bar with "File", "Edit", "Views", "Actions", "Reports", "Tools", and "Help". Below the menu is a toolbar with various icons including a gear, a blue arrow, a red X, a circular arrow, a printer, a green X, a blue W, a lock, an open lock, a curved arrow, a refresh icon, a question mark, and a green checkmark. The main area contains several fields: "Name" with the value "Part Related to a Project", "State" (empty), "Derived Relationship Family" with the value "Part Related to Project" and a blue ellipsis button to its right, "Root ItemType" (empty), and "Description" (empty text box).

Figure 35.

- Click . The DAC Definition screen appears with links to the Derived Relationship Family and the Root ItemType defined in the DR Family.

The screenshot shows a software window titled "Part Related to ...". It has a menu bar with "File", "Edit", "Views", "Actions", "Reports", "Tools", and "Help". Below the menu is a toolbar with various icons including a gear, a blue arrow, a red X, a circular arrow, a printer, a green X, a blue W, a lock, an open lock, a curved arrow, a refresh icon, a question mark, and a green checkmark. The main area contains several fields: "Name" with the value "Part Related to Project", "Derived Relationship Family" with the value "Part Related to Project" and a blue link icon to its left, "Root ItemType" with the value "Project" and a blue link icon to its left, and "Description" (empty text box).

Figure 36.

- Click the Editor icon  that appears in the left margin. A screen like the following appears. It highlights the Derived relationship associated with the DAC Definition.

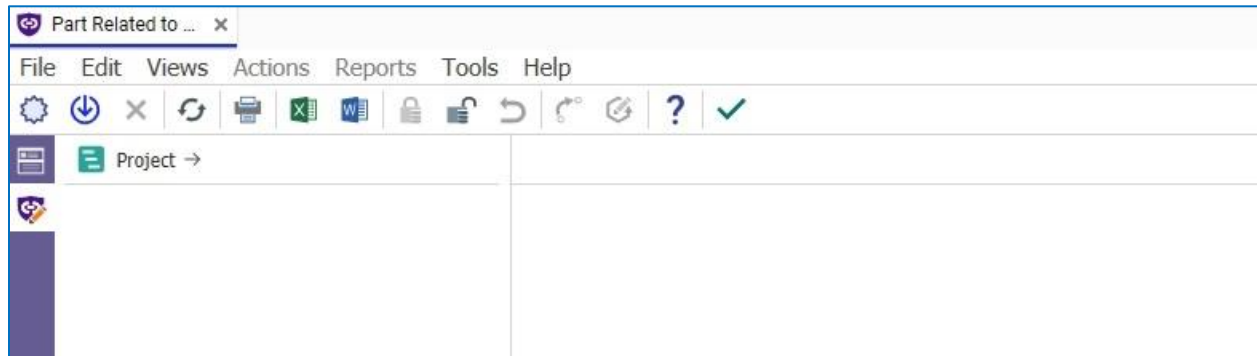


Figure 37.

- Click the **New Relationship** icon to specify permissions for the DAC Definition.

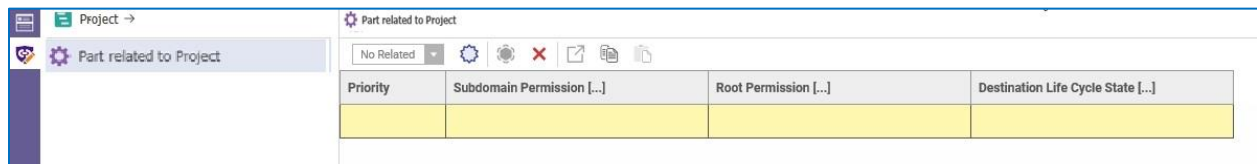


Figure 38.

- Enter a number in the **Priority** field.
- Click on the ellipses in the **Subdomain Permission** field and select a Permission. In Section [2.5](#), the example created was **Part related to Project**.

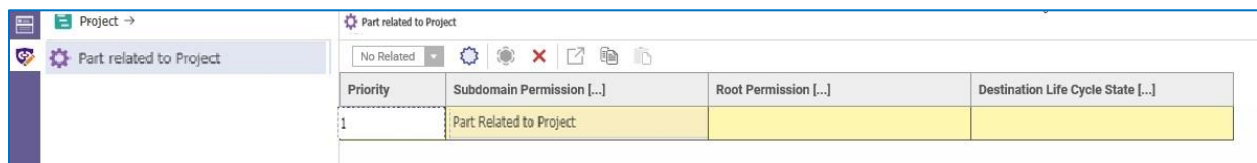


Figure 39.

- Click on the ellipses in the **Root Permission** field and select a Permission for the root. In this example, the Destination Life Cycle State is left empty to grant subdomain permissions to parts that have any Life Cycle state. If the Subdomain Policy should apply to a part in a specific Life Cycle State, click on the ellipses in the Destination Life Cycle State cell and select a Life Cycle State.

Note: A Subdomain Permission with an empty Destination Lifecycle State must have lower priority than a Subdomain Permission with a specified Destination Lifecycle with the same Root Permission.

- Repeat this process for all Subdomains that should be mapped.
- When complete, click **save** and then **unclaim** the DAC Definition.

2.7 Activating/Deactivating a DAC Definition

Once you have fully defined a DAC Definition and saved it, perform the following actions before activating it:

- Make sure that the underlying DR Family is active. If it isn't you will not be able to activate the DAC Definition.
- Validate the associated Subdomain policies.

The following procedure is an example of activating a DAC Definition:

1. Open the DAC definition and click the Editor button . A screen like the following appears:

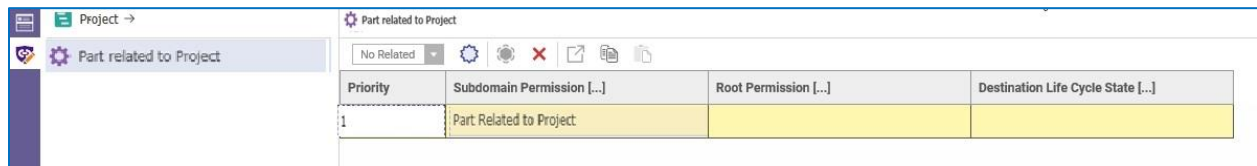


Figure 40.

2. Unclaim the DAC Definition and click the **Promote Item** icon. The Promote dialog box appears.

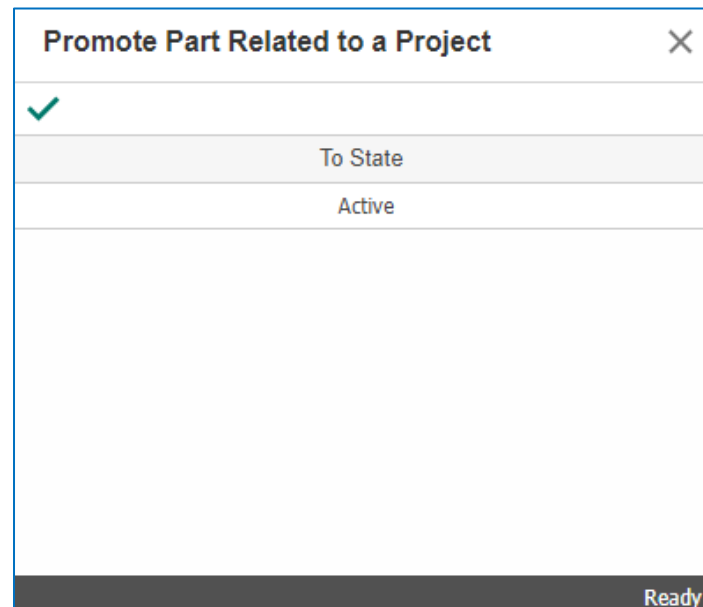


Figure 41.

3. Select **Active** and click the green checkmark. The DAC Definition is now in Active state and all permissions are propagated.

Appendix

Creating Teams

The following procedure is an example of creating a team to associate with an ItemType. For this example, we associate a Team to a Project.

1. From the Navigation pane, select **Administration>Teams**. The following menu appears:

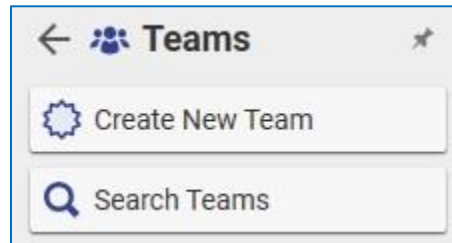


Figure 42.

2. Click **Create New Team**. The following window appears:

 A screenshot of a configuration window titled 'Team 1'. At the top, there's a tab labeled 'Team 1' with a close button. Below the title bar, there are three buttons: 'Save' (with a floppy disk icon), 'Done' (with a checkmark icon), and 'Delete'. The main content area has two sections. The first section, labeled 'Team', contains a 'Name' field (a large text box) and a 'Description' field (a smaller text box). The second section, labeled 'Team Members', contains a sub-section titled 'Identities' with a dropdown arrow and a star icon. Below this, there are several icons for adding, removing, and searching members, along with a 'Hidden' dropdown menu. At the bottom, there's a table with two columns: 'Name' and 'Role [...]'.

Figure 43.

- Enter the Team name and a brief description in the relevant fields.



- Click the **Select Items** icon to add team members. The Identity Search dialog appears:

Name	Description	Is Alias	Classification

< Prev Next > Page: 1 ... 25 Download

Figure 44.

- Click the **Search** icon to see a list of Identities. In this example, select Administrators and click the green arrow.



- Click **Done** to save and unclaim the item.

Creating Projects

The following procedure is an example of creating projects to associate with the teams created in the previous section:

- In the Navigation pane, select **Portfolio>Projects**. The following menu appears:

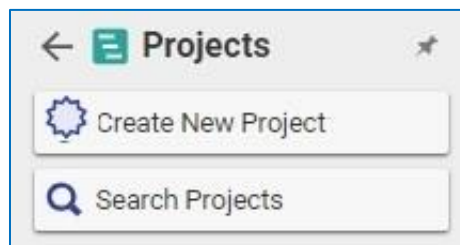
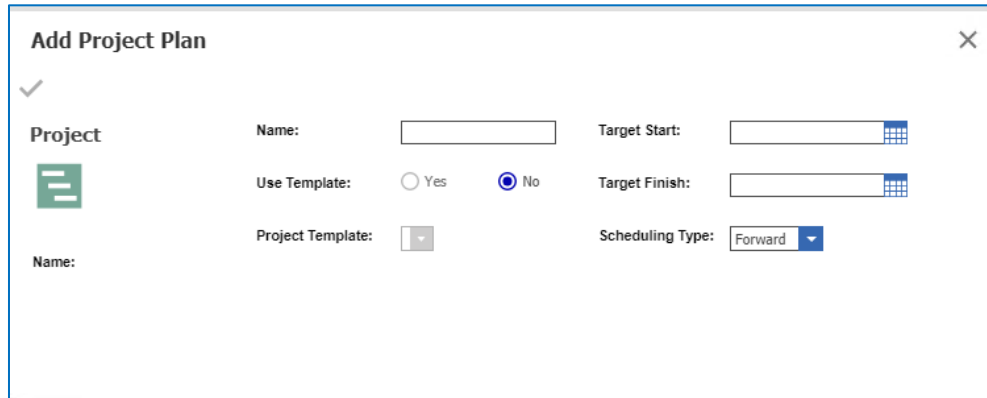


Figure 45.

- Click **Create New Project** to create a project to associate with one of the teams you created in the previous section. The Add Project Plan dialog box appears:

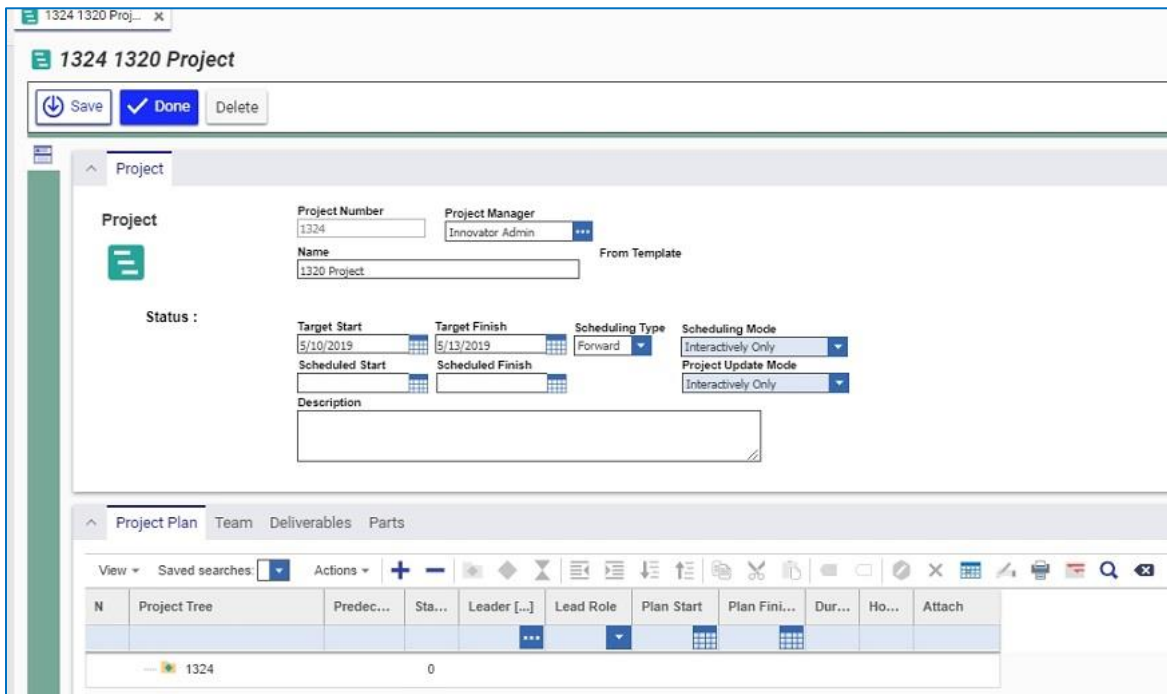


The 'Add Project Plan' dialog box contains the following fields and controls:

- Project** (icon)
- Name:** Text input field.
- Use Template:** Radio buttons for **Yes** and **No** (selected).
- Project Template:** Dropdown menu.
- Target Start:** Date picker.
- Target Finish:** Date picker.
- Scheduling Type:** Dropdown menu with **Forward** selected.

Figure 46.

- Enter the project name in the **Name** field.
- Click **Yes** if you want to use a project template. Select the template from the Project Template dropdown. If there are no existing templates, the Yes button and the Project Template dropdown are grayed out. Project1
- Click the calendar icon in the **Target Start** field to select the start date for the project. The **Target Finish** field automatically displays the next day. Click on the calendar icon to change the finish date.
- Specify the scheduling type by selecting either **Forward** or **Backward** from the Scheduling Type dropdown.
- Click the green arrow to create the project. The following screenshot shows an example of a project:



The screenshot shows the 'Project' details view for '1324 1320 Project'. The interface includes a top bar with 'Save', 'Done', and 'Delete' buttons. The main area displays the following information:

- Project Number:** 1324
- Project Manager:** Innovator Admin
- Name:** 1320 Project
- Status:** From Template
- Target Start:** 5/10/2019
- Target Finish:** 5/13/2019
- Scheduling Type:** Forward
- Scheduling Mode:** Interactively Only
- Scheduled Start:** (empty)
- Scheduled Finish:** (empty)
- Project Update Mode:** Interactively Only
- Description:** (empty text area)

Below the project details, there is a tabbed interface with 'Project Plan', 'Team', 'Deliverables', and 'Parts'. The 'Project Plan' tab is active, showing a table with columns: N, Project Tree, Predec..., Sta..., Leader [...], Lead Role, Plan Start, Plan Fini..., Dur..., Ho..., and Attach. The table currently shows one row with '1324' in the 'Project Tree' column and '0' in the 'Dur...' column.

Figure 47.

8. Click the ellipses in the **Project Manager** field. The Search dialog – Identity dialog box appears.

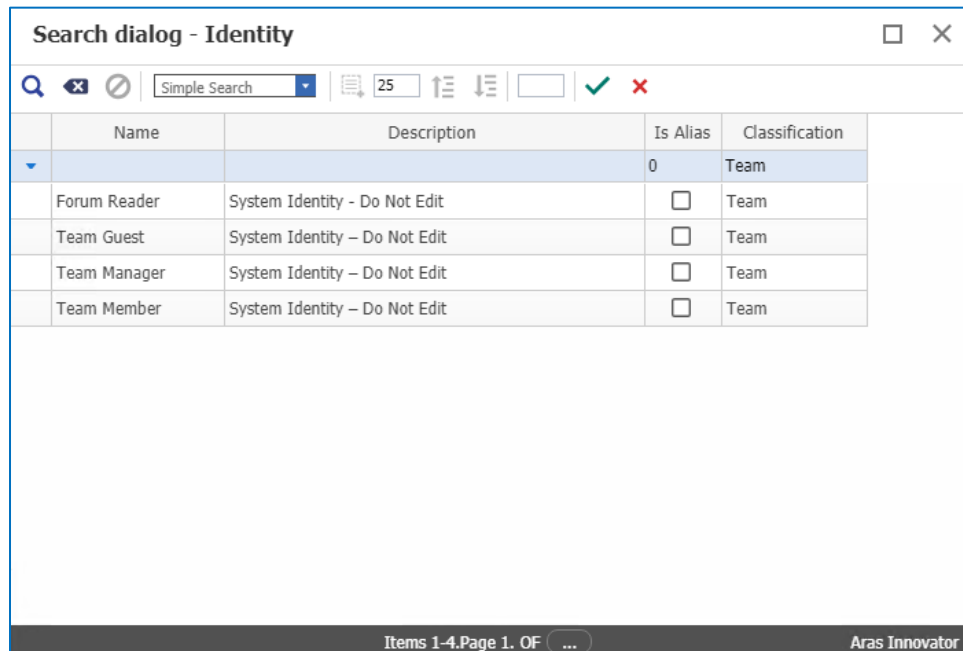


Figure 48.

9. Select **Team Manager** and click the green check icon.
10. Enter the project name, Target start/finish and Schedule start/finish dates in the appropriate fields.
11. Select the **Scheduling** and **Project Update** modes from the associated dropdown lists.
12. Select the **Team** tab and select either **Pick Related** or **No Related** from the dropdown menu. If you select Pick Related, the Search dialog box appears listing the available team roles.

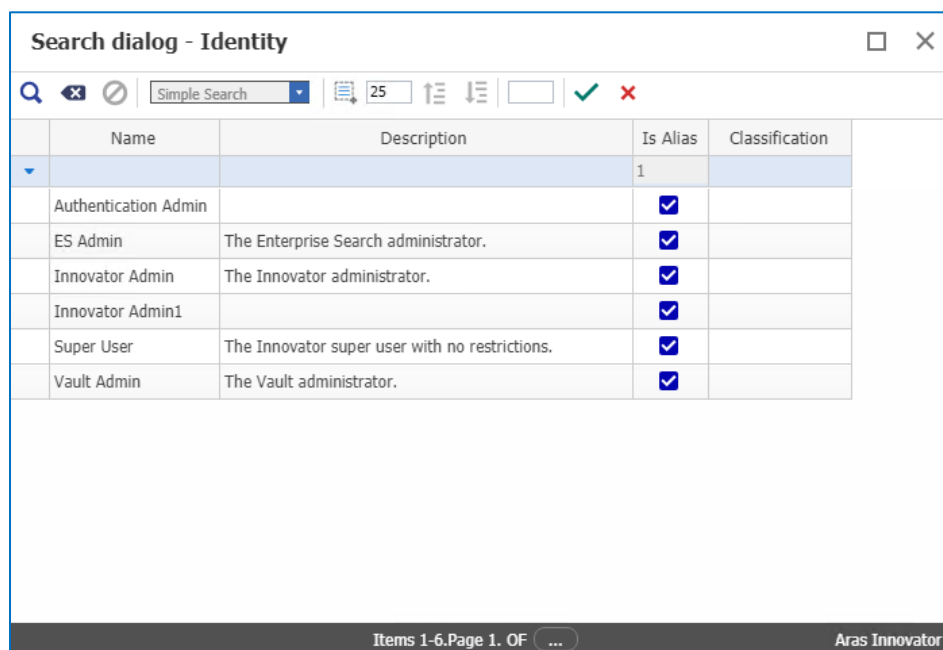


Figure 49.

13. Select the identity to associate with the Project and click the green arrow.

Associating Variables with a Project

1. Select the **Deliverables** tab to associate ItemTypes with the project.
2. Click **Select Items** to select existing ItemTypes. The Deliverable dialog box appears:

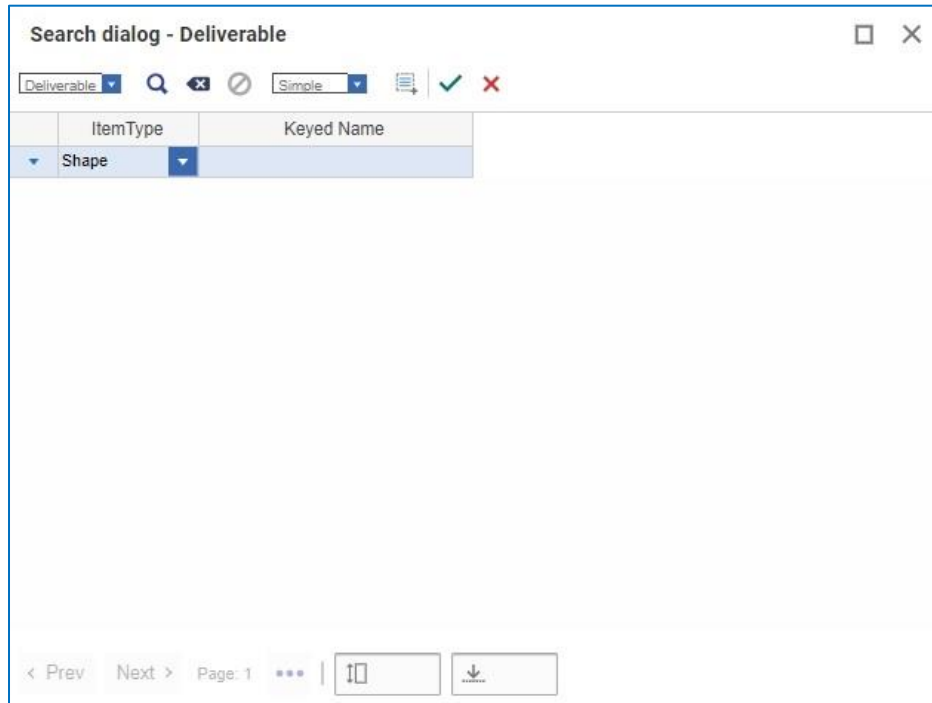


Figure 50.

3. Click the down arrow in the ItemType column and select an item from the list by clicking the green arrow.
4. To create ItemTypes to associate with the project, select **Create Items**. The Select ItemType dialog box appears:

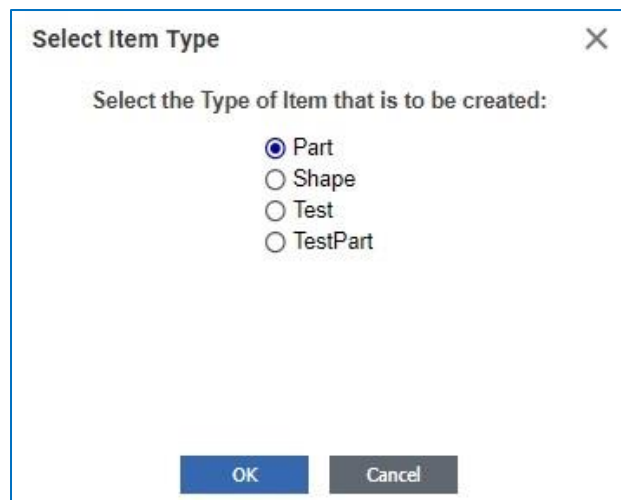


Figure 51.

5. Select the item type and click **OK**. The following screenshot shows a blank Part window:

Figure 52.

6. Enter a number in the **Part Number** field. The Revision and State fields are automatically populated when you save the part.
7. Click the ellipses in the **Assigned Creator** and **Designated User** fields to select roles to assign to the document.
8. Click the calendar icon to specify the **Effective Date** for the document.
9. Enter a description in the **Description** field.
10. Click the **Template** checkbox to use a template for the document.

Attaching Files to a Document

Use the following procedure to attach related files to a project document:

1. Click the **Add Documents** icon . A window similar to the following appears:

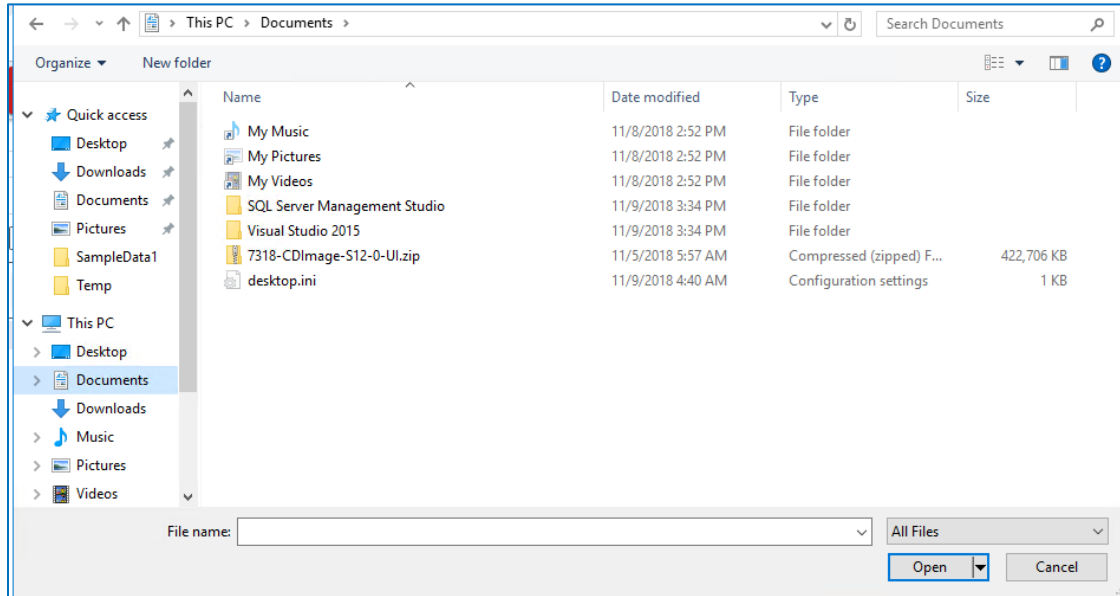


Figure 53.

2. Select the appropriate file and double-click on it to attach it to the document.
3. Click **Save**. The document appears on the Project Deliverables tab.
4. Click the **Part** tab and select the **New Relationship** icon to associate a Part with the project. The Search dialog box appears.

Adding Parts to a Project

Use the following procedure to add part information to a project:

1. Select the **Parts** tab to add one or more parts to a project.
2. Select **Pick Related** from the dropdown and click on the New Relationship tab to select existing parts. The Part search dialog appears.
3. Click the Run Search icon  to see a list of available parts:

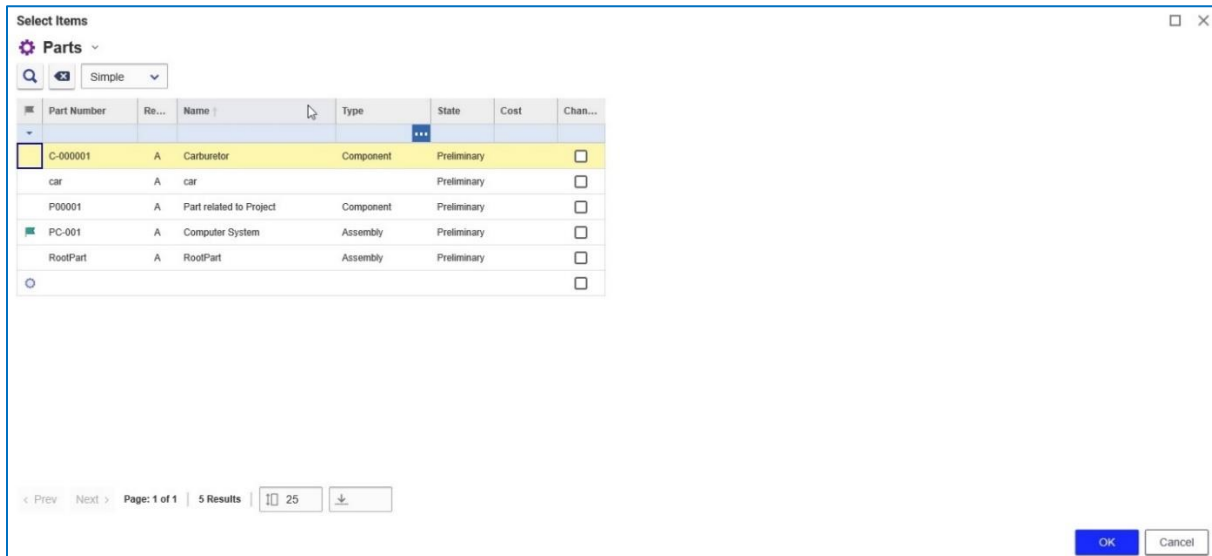


Figure 54.

4. Select the Parts that you want to attach to the project and click the green arrow. They are added to the Parts tab.
5. Select the Part and click the **Return Selected** icon. Click **Save**. Use this procedure to create a second project.
6. Associate the project with Team2.

Custom DRF Events Handling

Previously we discussed how DAC automatically maintains a record of every instance of every Derived Relationship in a table. It does so using an internal event handling mechanism which can also be leveraged and extended for configuration and customization of events in a DAC domain.

As of 12.0 SP5 an Event Handler is available under the DRF that provides *event types* and default methods OOTB. Events fire when item membership in a domain changes – for instance, when a Part item is added or removed from a BOM structure under an active DAC AC Policy. The event runs the associated method, executing business logic before `unidr_Relationships` are committed (persisted).

Note: The **Event Handlers** tab is hidden on the Derived Relationship Family UI by default.

To expose the **Event Handlers** relationship tab:

1. Log in as Admin.

2. Edit the **dr_RelationshipFamily** ItemType.
3. Set 'Default Structure View' to **Tabs On**.

The screenshot shows the configuration page for the **dr_RelationshipFamily** ItemType. At the top, there is a header with the item name and icons for Edit, Refresh, Undo, and other actions. Below the header, the configuration is organized into several sections:

- Name:** `dr_RelationshipFam`
- Singular Label:** `Derived Relationship`
- Plural Label:** `Derived Relationship`
- Show Parameters Tab:** `When Populated`
- History Template:** `Default`
- Default Structure View:** `Tabs On`
- Versioning:**
 - Versionable:** ☐
 - Discipline:** `Automatic`
 - Revisions:** `Default`
- Small Icon:** A small blue icon representing a relationship.
- Large Icon:** A larger blue icon representing a relationship.

Figure 55.

You will see the tab below when DRF is opened.

Aras provides 2 OOTB Event Handlers:

- **dr_HandleItemVersion** is a core function and should not be edited or removed.
- **dr_EnsurePropagatedAccess** checks whether adding a new item to the domain would provide permissions for the user that were not granted prior to the event.

The **dr_EnsurePropagatedAccess** Event Handler

To add an item to a Domain,

1. A user must have the **can_get** access right to the item. This handler is configured to enforce **can_get** OOTB; see the **Event Data** column in Figure 54.
2. An administrator can specify additional Access Types in a comma separated list, e.g., **can_get,can_delete**.

- If the user does not have any of the specified Access Types when adding an item to a Domain, an error is raised and the operation is rolled back.

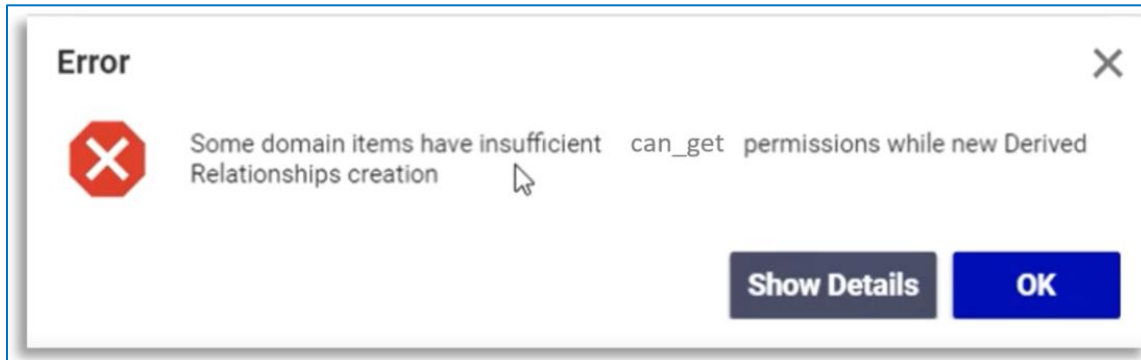


Figure 56.

The **OnBeforePersistRelationships** event will execute any method under the **Event Handler** column in **Execution Order**, with **Event Data** parameters if provided. Therefore, a knowledgeable implementor can provide an alternate method to **dr_EnsurePropagatedAccess** or even remove the Event entirely as business rules require.

Derived Relationships Event Handlers			
Event Handlers + - 🔍 ✖ Hidden 🔄 🖥 🌐			
Execution Or...	Event Type	Event Handler [...]	Event Data [...]
5	OnBeforePersistRelationships	dr_HandleItemVersion	
10	OnBeforePersistRelationships	dr_EnsurePropagatedAccess	<data><access_types>can_get</access_types></data>

Figure 57.